

*Développement d'une Application Mobile de
Reconnaissance et de Classification d'Objets avec
IA*

Rapport de SAÉ présenté en janvier 2026

par

*Théo SCHALLER
Loric BONDON
Baptiste BRODIER*

BUT Informatique - 3^e Année

Académie de Nancy-Metz
Université de Lorraine
Institut Universitaire Technologique de Metz
Département Informatique
Promotion 2023-2026

Table des matières

Table des matières.....	2
1. Introduction et contexte du projet.....	4
1.1 Contexte de la SAÉ.....	4
1.2 Les 7 émotions du FER-2013.....	4
1.3 Aperçu de l'application.....	4
2. Choix techniques et justifications.....	5
2.1 Pourquoi Android et Kotlin ?.....	5
2.2 TensorFlow Lite pour l'IA.....	5
2.3 ML Kit pour la détection de visages.....	5
2.4 Architecture offline-first avec Room Database.....	6
2.5 Backend Spring Boot.....	6
3. Architecture de l'application.....	7
3.1 Diagramme de cas d'utilisation.....	7
3.2 Vue d'ensemble de l'architecture.....	7
3.3 Flux de données pour la détection en temps réel.....	8
3.4 Schéma de la base de données.....	8
4. Le modèle d'intelligence artificielle.....	9
4.1 Le dataset FER-2013.....	9
4.2 Entraînement du modèle.....	9
4.3 Performances et limites.....	9
4.4 Limites du dataset et de l'approche.....	10
5. Fonctionnalités développées.....	11
5.1 Écran d'accueil.....	11
5.2 Détection en temps réel et résultats.....	11
5.3 Historique des détections.....	12
5.4 Profil et synchronisation.....	12
5.5 Paramètres.....	13
5.6 Entraînement personnalisé.....	13

6. Difficultés rencontrées et solutions.....	14
6.1 Le problème du cadrage des visages.....	14
6.2 Instabilité des prédictions en temps réel.....	14
6.3 Modèle trop lourd.....	14
6.4 Migration Firebase vers Spring Boot.....	14
6.5 Complexité de l'entraînement personnalisé.....	15
6.6 Passage de données JSON au script Python.....	15
7. DevOps et qualité de code.....	16
7.1 Intégration continue avec GitHub Actions.....	16
7.2 Analyse SonarCloud.....	16
7.3 Organisation Git.....	16
7.4 Tests.....	17
8. Bilan et perspectives.....	18
8.1 Objectifs atteints.....	18
8.2 Ce qu'on a appris.....	18
8.3 Améliorations possibles.....	18
8.4 Conclusion.....	19
9. Annexes.....	20
9.1 Liens du projet.....	20
9.2 Stack technique complète.....	20
9.3 Répartition du travail.....	20

1. Introduction et contexte du projet

1.1 Contexte de la SAÉ

La SAÉ 5.01 nous demandait de créer une application mobile capable de reconnaître et classer des objets en temps réel avec de l'intelligence artificielle. On avait beaucoup de liberté sur le choix du sujet, tant que ça restait dans le domaine de la vision par ordinateur.

Notre groupe a choisi de travailler sur la reconnaissance d'expressions faciales. L'idée était de faire une app qui puisse analyser le visage de quelqu'un via la caméra et détecter son expression : joie, tristesse, colère, surprise, peur, dégoût ou neutre. On a appelé notre projet MoodScan.

On a donc sélectionné le dataset FER-2013 (Facial Expression Recognition 2013), assez connu et bien documenté, ce qui nous facilitait le travail côté données d'entraînement.

1.2 Les 7 émotions du FER-2013

Le dataset FER-2013 contient 35 887 images en niveaux de gris de 48x48 pixels, réparties en 7 catégories d'expressions :

1. Joie (Happy)
2. Tristesse (Sad)
3. Colère (Angry)
4. Surprise (Surprise)
5. Peur (Fear)
6. Dégoût (Disgust)
7. Neutre (Neutral)

Sachant que ce dataset est connu pour être difficile. Même des humains font des erreurs dessus, et les meilleurs modèles plafonnent autour de 77% de précision. Dans notre cas, nos 60-65% sont en fait pas si mal que ça.

1.3 Aperçu de l'application

Voici la maquette Figma réalisée en début de projet pour définir l'interface de l'application :

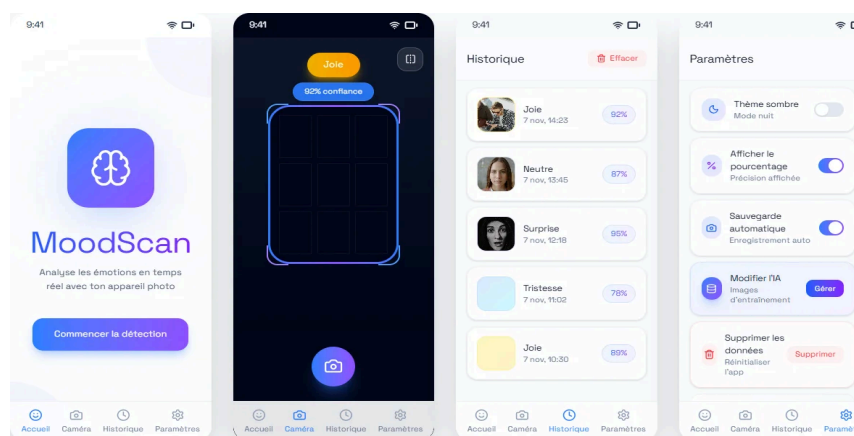


Figure 1 : Maquette Figma de l'application

2. Choix techniques et justifications

2.1 Pourquoi Android et Kotlin ?

On a choisi Android plutôt qu'iOS pour plusieurs raisons assez pratiques. Déjà, on avait tous des téléphones Android pour tester. Ensuite, le déploiement et le debug sont plus simples, pas besoin de Mac ni de compte développeur Apple payant.

Pour le langage, on est partis sur Kotlin plutôt que Java ou Flutter. C'est le langage recommandé par Google pour le développement Android moderne, et il est moins verbeux que certains autres langages. Ça nous a demandé un petit temps d'adaptation au début parce qu'on ne connaissait pas vraiment, mais au final la syntaxe est assez intuitive.

Pour l'interface, on utilise Jetpack Compose, le toolkit de Google. C'est plus moderne que les layouts XML traditionnels et ça permet de créer des interfaces réactives plus facilement.

2.2 TensorFlow Lite pour l'IA

Pour faire tourner notre modèle d'IA sur le téléphone, on utilise TensorFlow Lite (TFLite). C'est une version optimisée de TensorFlow conçue spécifiquement pour les appareils mobiles et embarqués.

L'avantage principal c'est que tout le traitement se fait en local sur le téléphone, pas besoin d'envoyer les images vers un serveur distant. Ça veut dire que l'app fonctionne même sans connexion internet, et ça respecte mieux la vie privée des utilisateurs étant donné que la plupart des scans restent sur leur appareil.

Par contre, ça implique des contraintes : le modèle doit être suffisamment léger pour ne pas faire ramer le téléphone. On a dû faire des compromis entre la taille du modèle et sa précision.

Pour reconnaître les expressions faciales, on utilise un CNN (réseau de neurones convolutif) qu'on a entraîné nous-mêmes sur le dataset FER-2013. On a pas utilisé YOLO parce que c'est fait pour détecter plusieurs objets en même temps avec leur position, alors que nous on a juste besoin de classifier une émotion sur un visage déjà détecté. MobileNet non plus parce que c'est prévu pour des images en couleur 224×224 avec plein de catégories différentes, c'est trop lourd pour ce qu'on veut faire. Du coup on a préféré partir sur une architecture CNN maison, plus légère et adaptée à nos images 48×48 en noir et blanc.

2.3 ML Kit pour la détection de visages

Avant de pouvoir analyser une expression, il faut d'abord détecter où se trouve le visage dans l'image. Pour ça, on utilise ML Kit de Google, qui fournit une API prête à l'emploi pour la détection de visages.

Le processus fonctionne en deux étapes : d'abord ML Kit détecte et localise les visages dans chaque frame de la caméra, puis on extrait la zone du visage et on l'envoie à notre modèle TFLite pour l'analyse de l'expression.

2.4 Architecture offline-first avec Room Database

Une décision importante qu'on a prise assez tôt dans le projet : la base de l'application doit offrir un contrôle total sur les données utilisateur. On utilise donc Room Database (la bibliothèque de persistance d'Android) pour stocker localement l'historique des détections et les préférences utilisateur.

Ce stockage local est destiné aux utilisateurs connectés : il leur permet de rester maîtres de leurs photos et de décider eux-mêmes quand les partager avec le serveur backend. La synchronisation n'est jamais automatique, c'est l'utilisateur qui choisit ce qu'il envoie. Ce choix nous a semblé plus cohérent : pourquoi imposer un envoi systématique alors qu'on peut laisser chacun gérer ses données à son rythme ?

2.5 Backend Spring Boot

Pour les fonctionnalités qui nécessitent un serveur (authentification, stockage cloud, entraînement personnalisé), on a développé un backend avec Spring Boot et une base de données MySQL.

Au départ, on était partis sur Firebase parce que c'était plus simple à mettre en place. Mais on s'est vite rendu compte que le stockage d'images sur Firebase Storage devenait payant assez rapidement, et que ça limitait nos possibilités pour l'entraînement personnalisé. Du coup, on a fait une migration complète vers une architecture plus classique avec Spring Boot.

L'authentification se fait via JWT (JSON Web Tokens), avec les mots de passe hashés en BCrypt.

3. Architecture de l'application

3.1 Diagramme de cas d'utilisation

Voici le diagramme de cas d'utilisation UML qui décrit les différentes interactions possibles entre l'utilisateur et l'application :

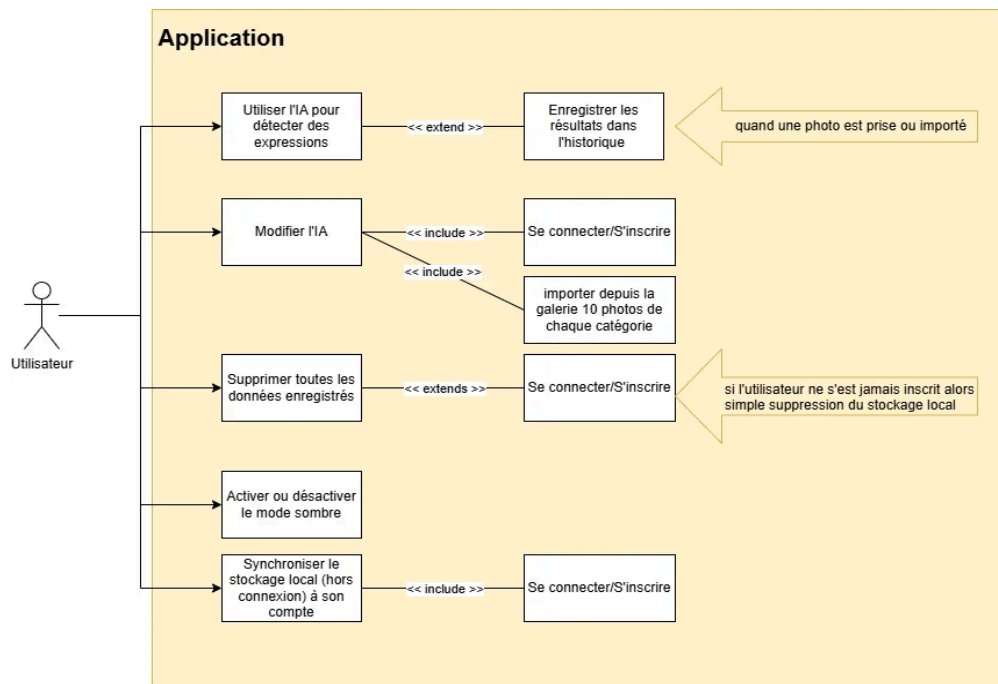


Figure 2 : Diagramme de cas d'utilisation de l'application

On distingue les fonctionnalités accessibles à tous (détection, historique local, paramètres) de celles qui nécessitent une authentification (modification de l'IA, synchronisation cloud). La suppression des données fonctionne différemment selon que l'utilisateur est connecté ou non.

3.2 Vue d'ensemble de l'architecture

L'architecture de MoodScan suit un pattern assez classique pour une application Android moderne, avec une séparation claire entre les différentes couches.

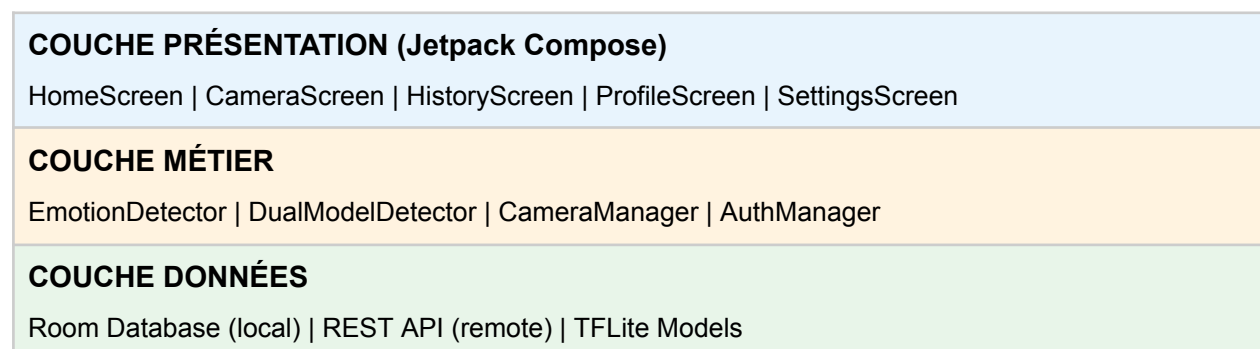


Figure 3 : Architecture en couches de l'application

3.3 Flux de données pour la détection en temps réel

Le processus de détection d'expression suit ce flux :

1. CameraX capture une frame vidéo
2. ML Kit détecte les visages présents dans la frame
3. Pour chaque visage détecté, on extrait la région correspondante
4. L'image est convertie en niveaux de gris et redimensionnée en 48x48 pixels
5. Le modèle TFLite analyse l'image et retourne les probabilités pour chaque émotion
6. L'émotion avec la plus haute probabilité est affichée à l'écran (pour éviter les variations trop rapides, on applique un lissage temporel : on calcule la moyenne des 5 dernières prédictions, et on ne traite qu'une image toutes les 3 frames).

3.4 Schéma de la base de données

Voici le schéma entité-association (modèle Merise) de notre base de données :

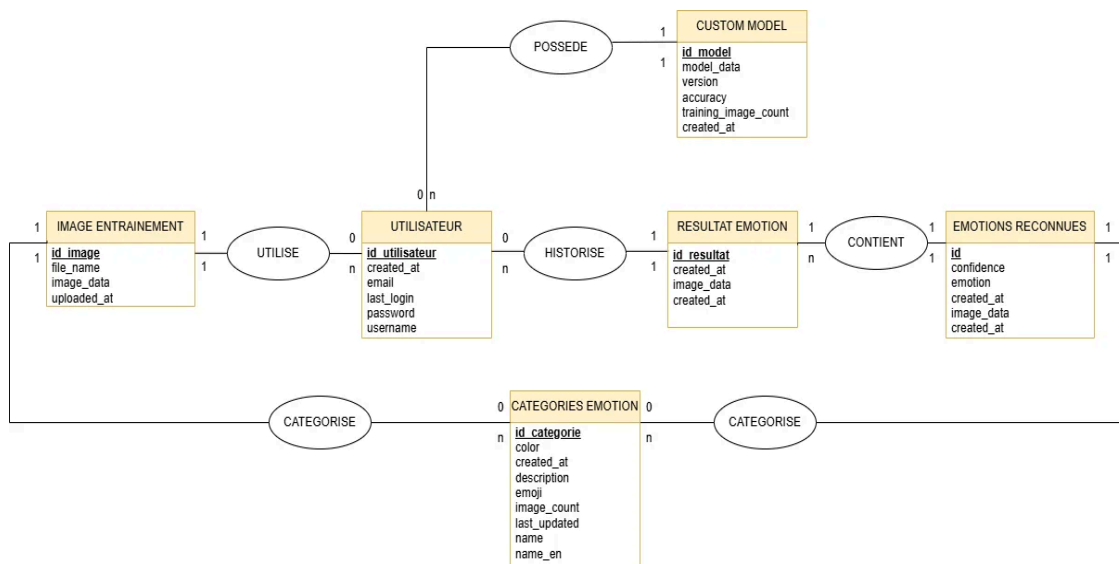


Figure 4 : Schéma entité-association de la base de données

Les principales entités sont :

- **UTILISATEUR** : informations de compte (email, username, mot de passe hashé)
- **CATEGORIES_EMOTION** : les 7 catégories d'émotions avec leurs métadonnées (nom FR/EN, couleur, emoji)
- **IMAGE_ENTRAINEMENT** : images uploadées par les utilisateurs pour l'entraînement personnalisé
- **RESULTAT_EMOTION** : historique des détections
- **EMOTION_RECONNUES** : ensemble des émotions détectés dans une capture
- **CUSTOM_MODEL** : modèles personnalisés générés (fichier .tflite, version, précision)

4. Le modèle d'intelligence artificielle

4.1 Le dataset FER-2013

Le FER-2013 (Facial Expression Recognition 2013) est un dataset créé pour un challenge Kaggle. Il contient 35 887 images de visages en niveaux de gris, toutes normalisées à 48x48 pixels.

Répartition des images par émotion :

Émotion	Nombre d'images
Joie (Happy)	8 989 (~25%)
Neutre (Neutral)	6 198 (~17%)
Tristesse (Sad)	6 077 (~17%)
Peur (Fear)	5 121 (~14%)
Colère (Angry)	4 953 (~14%)
Surprise (Surprise)	4 002 (~11%)
Dégoût (Disgust)	547 (~1.5%)

On voit tout de suite un problème : le déséquilibre des classes. Il y a 16 fois plus d'images de joie que de dégoût. Ça explique en partie pourquoi certaines émotions sont mieux reconnues que d'autres.

4.2 Entraînement du modèle

L'entraînement du modèle se fait en Python avec TensorFlow sur Google Colab. On a choisi Colab parce que ça nous donne accès à des GPU gratuitement, et parce que l'entraînement d'un modèle de deep learning sur CPU est très long.

Notre architecture de réseau de neurones est un CNN (Convolutional Neural Network) relativement simple avec plusieurs couches de convolution suivies de couches de pooling, puis des couches fully connected à la fin. On a ajouté du Dropout pour éviter l'overfitting.

Pour l'augmentation de données du modèle personnalisé (spécifique à chaque utilisateur), on applique des transformations aléatoires aux images d'entraînement : rotation, zoom, décalage horizontal/vertical, et mode miroir. Ça permet de multiplier artificiellement le nombre d'images par 5 environ et d'avoir un modèle qui généralise mieux.

4.3 Performances et limites

On a testé deux versions de notre modèle :

- Modèle léger (833 KB) : ~60% de précision, fonctionne de manière fluide sur tous les téléphones testés
- Modèle lourd (7 MB) : ~65% de précision, mais causait des freezes sur certains téléphones

On a choisi de garder le modèle léger par défaut parce que la différence de 5% ne justifie pas les problèmes de performance. L'expérience utilisateur passe avant tout.

La joie (Happy) est de loin l'émotion la mieux reconnue avec environ 90% de précision. C'est logique étant donné que c'est la catégorie avec le plus d'images et que les sourires sont des expressions assez distinctives. Par contre, le dégoût et la peur sont souvent confondus entre eux ou avec d'autres émotions.

4.4 Limites du dataset et de l'approche

Il faut être honnête sur les limitations de notre système :

- Ambiguïté des expressions : même des humains n'arrivent qu'à environ 65% de précision sur ce dataset. Certaines images sont vraiment ambiguës.
- Biais du dataset : FER-2013 contient surtout des visages occidentaux, ce qui peut affecter la reconnaissance sur d'autres populations.
- Expressions vs Émotions : dans un premier temps, on avait pour projet de détecter les émotions sauf que techniquement, on détecte des expressions faciales, pas des émotions. Quelqu'un peut sourire sans être vraiment content.
- Conditions réelles : le modèle a été entraîné sur des images frontales bien cadrées. En conditions réelles avec de l'éclairage variable et des angles différents, les performances baissent.

5. Fonctionnalités développées

5.1 Écran d'accueil

L'écran d'accueil présente l'application avec le logo MoodScan et un bouton pour lancer directement la détection. La navigation se fait via une barre en bas de l'écran avec 4 onglets : Accueil, Caméra, Historique et Profil.

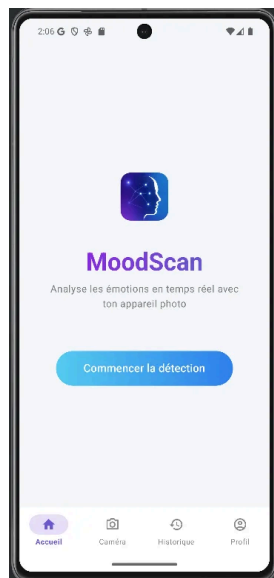


Figure 5a : Écran d'accueil

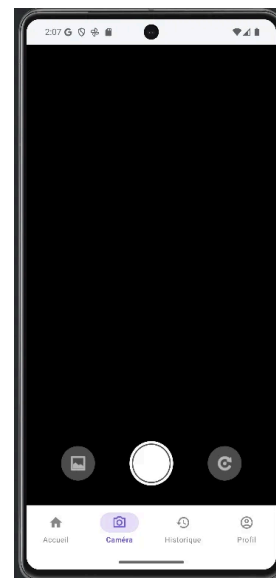


Figure 5b : Écran caméra

5.2 Détection en temps réel et résultats

C'est la fonctionnalité principale de l'application. Quand on ouvre l'écran caméra, l'app analyse le flux vidéo en continu. On peut capturer une photo pour obtenir une analyse détaillée avec le pourcentage de confiance. Le résultat s'affiche dans une modale avec l'émotion détectée et une barre de progression.

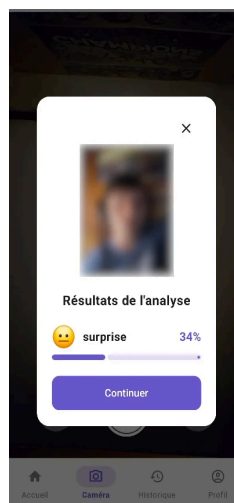


Figure 6 : Modale d'affichage des résultats de détection

5.3 Historique des détections

Toutes les détections sont sauvegardées localement dans la Room Database avec l'image, l'émotion détectée, le pourcentage de confiance et la date. Les utilisateurs peuvent consulter leur historique sous forme de cartes avec un aperçu de chaque analyse.

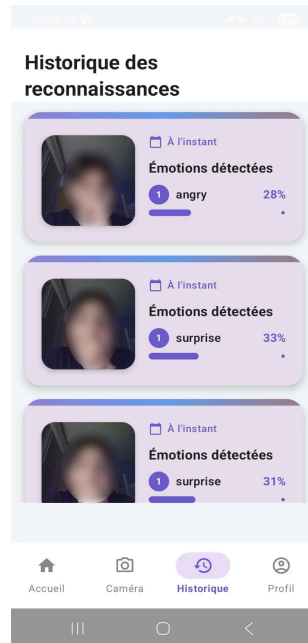


Figure 7 : Historique des reconnaissances

5.4 Profil et synchronisation

Les utilisateurs peuvent créer un compte pour débloquer les fonctionnalités cloud. L'écran de profil affiche des statistiques (nombre total de détections, synchronisées, locales) et permet de synchroniser les données avec le serveur.

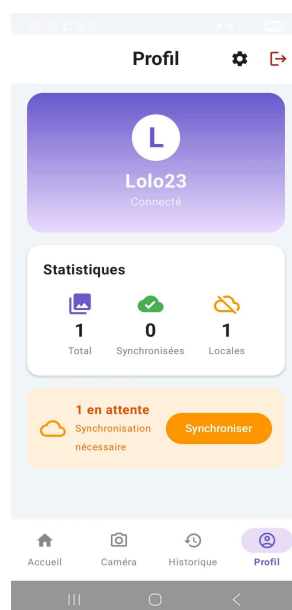


Figure 8 : Écran de profil avec statistiques et synchronisation

5.5 Paramètres

L'écran des paramètres propose plusieurs options : thème sombre/clair, affichage du pourcentage de confiance, sauvegarde automatique, accès à la gestion de l'IA, et réinitialisation des données.

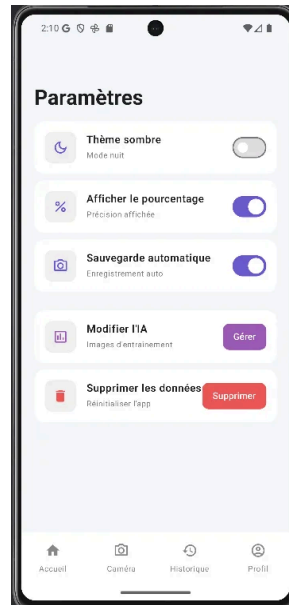


Figure 9 : Écran des paramètres

5.6 Entraînement personnalisé

C'est la feature la plus avancée de l'app. Les utilisateurs connectés peuvent uploader leurs propres photos pour chaque catégorie d'émotion (minimum 10 par catégorie), puis lancer un entraînement pour créer un modèle personnalisé. L'écran montre la précision du modèle actuel et le nombre d'images par catégorie.

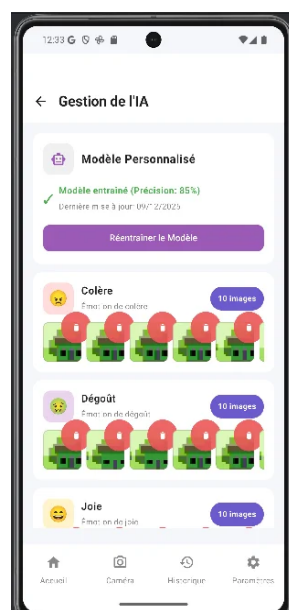


Figure 10 : Écran de gestion de l'IA et entraînement personnalisé

6. Difficultés rencontrées et solutions

6.1 Le problème du cadrage des visages

Un des premiers bugs qu'on a eu : ML Kit détectait les visages mais le cadrage était souvent décalé. On récupérait juste une partie du visage (ex: la mâchoire et une joue) au lieu du visage entier. Forcément, le modèle ne pouvait pas bien analyser avec des données partielles.

On a donc ajouté une marge de 20% autour de la boîte retournée par ML Kit. Ce n'est pas parfait dans tous les cas, mais ça améliore beaucoup la qualité des images envoyées au modèle.

6.2 Instabilité des prédictions en temps réel

Au début, l'émotion affichée changeait tout le temps, même quand on gardait la même expression. Un coup "Joie", un coup "Neutre", un coup "Surprise"... C'était vraiment pas utilisable.

On a donc implémenté un système de lissage temporel. Au lieu d'afficher la prédiction brute de chaque frame, on calcule la moyenne des 5 dernières prédictions. En plus, on ne traite qu'une frame sur 3 pour réduire la charge CPU. Maintenant l'affichage est beaucoup plus stable.

6.3 Modèle trop lourd

Quand on a voulu utiliser notre modèle à 65% de précision (7 MB), l'app se mettait à ramer sur certains téléphones. Sur les plus anciens, ça freezait carrément pendant quelques secondes à chaque détection.

On est donc revenus au modèle léger (833 KB, 60% de précision). On a préféré sacrifier un peu de précision pour avoir une expérience fluide. C'est frustrant mais c'est le compromis réaliste sur mobile.

6.4 Migration Firebase vers Spring Boot

On avait commencé avec Firebase parce que c'était rapide à mettre en place, sur un serveur distant et facile d'accès. Mais quand on a voulu stocker les images d'entraînement des utilisateurs, on s'est rendu compte que Firebase Storage devenait payant très vite. Et puis pour l'entraînement personnalisé, on avait besoin d'un vrai serveur pour exécuter le script Python.

On a donc fait une refonte complète en migrant vers Spring Boot avec MySQL. Ça a pris pas mal de temps mais maintenant on a un contrôle total sur notre backend et on peut faire tourner n'importe quel traitement côté serveur.

6.5 Complexité de l'entraînement personnalisé

Au départ, on voulait faire l'entraînement directement sur le téléphone. Mauvaise idée : il aurait fallu décompresser tout le dataset FER-2013 sur l'appareil (plusieurs centaines de MB), et le temps d'entraînement aurait été de plusieurs heures sur un CPU mobile.

L'entraînement se fait donc côté serveur. L'utilisateur upload ses images, le backend lance le script Python, et le modèle généré est ensuite téléchargeable. C'est plus long en termes de transfert réseau mais au moins c'est faisable (on risque par contre d'avoir un problème si un grand nombre d'utilisateurs décide d'entraîner son modèle au même moment).

6.6 Passage de données JSON au script Python

Un bug bête qu'on a mis du temps à comprendre : quand on passait les données d'images en JSON directement en argument de ligne de commande au script Python, ça plantait avec des datasets un peu gros. En fait, il y a une limite de longueur sur les arguments en ligne de commande sous Windows.

Pour résoudre ce problème, au lieu de passer le JSON en argument, on l'écrit dans un fichier temporaire et on passe juste le chemin du fichier au script Python.

7. DevOps et qualité de code

7.1 Intégration continue avec GitHub Actions

On a mis en place trois workflows GitHub Actions qui se déclenchent automatiquement à chaque push ou pull request :

1. Android CI : compile l'APK debug, exécute les tests unitaires, lance le lint Android pour détecter les erreurs de style et bugs potentiels, et scanne les vulnérabilités des dépendances (Retrofit, TensorFlow Lite, CameraX, etc.).
2. Backend CI : compile le JAR Spring Boot, lance les tests backend, et scanne les vulnérabilités des dépendances serveur (Spring Boot, JWT, MySQL connector).
3. SonarCloud Analysis : analyse approfondie du code Kotlin/Java avec détection de bugs, code smells, vulnérabilités de sécurité, et calcul de la complexité cyclomatique.

7.2 Analyse SonarCloud

SonarCloud nous génère un dashboard web avec plein d'indicateurs sur la qualité du code. C'est super utile pour identifier les points à améliorer en priorité.

Les principales alertes qu'on a eues concernaient des problèmes de sécurité autour de l'accès au stockage externe et de la vérification des dépendances Gradle. C'était une recommandation d'activer la vérification des signatures, mais ça ajoutait pas mal de complexité pour un projet académique donc on a documenté le risque sans l'implémenter.

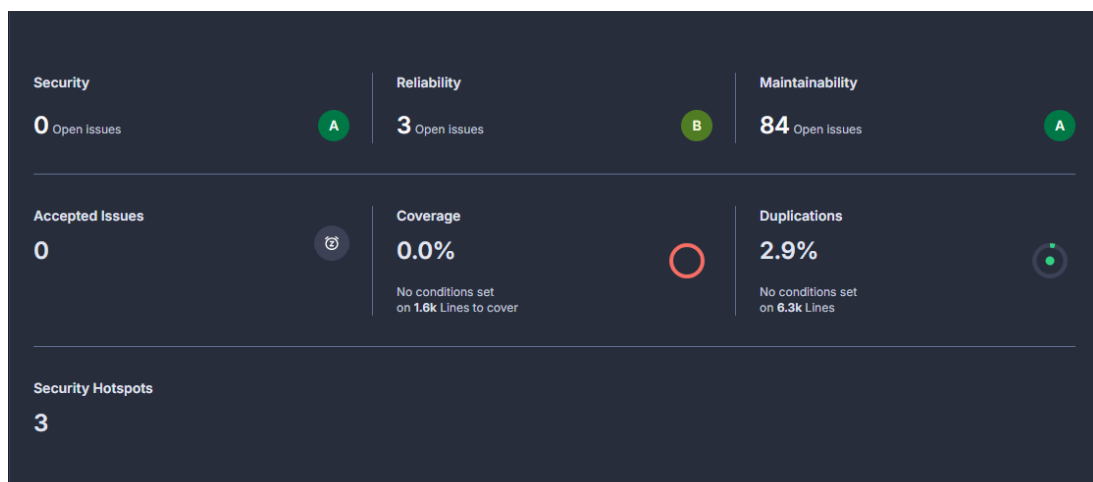


Figure 11 : Dashboard SonarCloud (coverage à 0% puisque non implémenté)

7.3 Organisation Git

On a utilisé GitHub pour le versioning avec une organisation assez simple : une branche main pour le code stable, et des branches feature pour les développements en cours. Chaque vendredi on faisait un push avec un commit qui correspond au rapport hebdomadaire. On a aussi utilisé Trello pour la gestion de projet avec un tableau Kanban classique (To Do / In Progress / Done).

7.4 Tests

Honnêtement, les tests c'est le point faible du projet. On a quelques tests unitaires basiques mais la couverture du code reste très faible. La majorité des tests ont été faits manuellement sur nos téléphones.

C'est quelque chose qu'on améliorerait si on avait plus de temps : ajouter des tests d'intégration pour l'API REST, des tests UI avec Espresso, et augmenter la couverture des tests unitaires.

8. Bilan et perspectives

8.1 Objectifs atteints

Si on regarde les fonctionnalités, on a :

1. Acquisition d'images : flux vidéo en temps réel via CameraX + import depuis la galerie
2. Reconnaissance en temps réel : modèle TFLite fonctionnel avec affichage des résultats en overlay sur la caméra
3. Base de catégories : les 7 émotions FER-2013
4. Apprentissage personnalisé : système complet pour entraîner un modèle avec ses propres images
5. Interface utilisateur : design moderne avec Jetpack Compose, thème sombre/clair, navigation intuitive
6. Fonctionnalités supplémentaires : historique local, synchronisation cloud optionnelle, profil utilisateur avec statistiques

8.2 Ce qu'on a appris

Ce projet nous a permis de toucher à pas mal de technologies qu'on ne connaissait pas avant :

- Kotlin et Jetpack Compose pour le développement Android moderne
- TensorFlow et TensorFlow Lite pour l'IA embarquée
- Les contraintes du déploiement mobile (taille des modèles, performance, batterie)
- CI/CD avec GitHub Actions et analyse de code avec SonarCloud
- Architecture offline-first et synchronisation cloud

On a aussi appris à faire des compromis réalistes. Par exemple accepter 60% de précision si ça permet d'avoir une app fluide, ou sacrifier une fonctionnalité si elle complexifie trop l'architecture.

8.3 Améliorations possibles

Si on devait continuer le projet, voici ce qu'on améliorerait :

1. Améliorer le modèle : tester des architectures plus sophistiquées comme EfficientNet ou MobileNetV3 qui sont optimisées pour le mobile
2. Plus de tests : augmenter significativement la couverture de tests automatisés
3. iOS : porter l'application sur iOS avec Core ML/Kotlin MultiPlatform pour toucher plus d'utilisateurs
4. Statistiques avancées : ajouter des graphiques d'évolution des émotions dans le temps
5. Créer de l'interaction avec les utilisateurs

8.4 Conclusion

MoodScan est une application fonctionnelle qui remplit les objectifs fixés par la SAÉ 5.01. Elle permet de détecter des expressions faciales en temps réel sur un téléphone Android, avec une architecture technique solide et des fonctionnalités bonus.

On est contents du résultat même si on voit bien les limites. La précision de 60% c'est correct pour du FER-2013 mais c'est pas suffisant pour des applications critiques. Et il reste encore du travail côté tests et documentation.

En tout cas, ce projet nous a donné une bonne expérience concrète sur le développement d'applications IA mobiles, de l'entraînement du modèle jusqu'au déploiement, en passant par tous les problèmes qu'on peut rencontrer en chemin.

9. Annexes

9.1 Liens du projet

- **GitHub** : <https://github.com/LoricBndn/SAE5.01>
- **Trello** : <https://trello.com/b/GYnaCKyV/sae501>

9.2 Stack technique complète

Composant	Technologies
Application mobile	Android, Kotlin, Jetpack Compose
IA / ML	TensorFlow, TensorFlow Lite, ML Kit
Base de données locale	Room Database (SQLite)
Backend	Spring Boot (Java), MySQL
Authentification	JWT, BCrypt
Réseau	Retrofit, REST API
Caméra	CameraX
CI/CD	GitHub Actions
Qualité de code	SonarCloud, Android Lint
Entraînement modèles	Python, Google Colab, Jupyter

9.3 Répartition du travail

Membre	Contributions principales
Loric BONDON	Interface utilisateur (reproduction maquette et améliorations visuelles), page historique (UI/UX + backend), page profil (UI/UX + backend), stockage local (base de données locale), préférences de l'app dans settings (mode sombre, affichage pourcentage, sauvegarde auto)
Baptiste BRODIER	Interface utilisateur, intégration Compose, page caméra (UX : boutons, prendre en photo, modale de détail de détection)
Théo SCHALLER	Modèle IA, entraînement TensorFlow, DevOps (CI/CD), architecture backend, API REST, base de données, système d'authentification, page "Gérer l'IA", documentation, maintenance et maquette